==========

==========

# CHAPTER 19 — MMU PERFORMANCE OPTIMIZATION

==========

==========

***A maximum-detail, RTOS/QNX-style technical breakdown of how to optimize ARMv8 MMU performance across TLB behavior, page-table design, cacheability, descriptor layout, Stage-1/Stage-2 interactions, and SMP systems.***

**********

**********

## 19.1 — Introduction to MMU Performance Optimization

**********

**********

MMU performance determines:

- memory access latency
- TLB hit rate
- page-table walk frequency
- cache efficiency
- SMP scalability
- virtualization overhead
- DMA throughput
- device access latency

Optimizing MMU performance requires:

- correct granule selection
- correct block/page usage
- correct TLB behavior
- correct descriptor layout
- correct cacheability
- correct shareability
- correct Stage-2 design

This chapter provides **practical, architecture-level optimization strategies**.

**********

**********

# 19.2 — Performance Bottlenecks in MMU Systems

**********

**********

Major MMU bottlenecks include:

### TLB Misses

- cause page-table walks
- increase latency
- increase memory traffic

### Page-Table Walks

- require multiple memory accesses
- may miss in cache
- may miss in walk cache

### Descriptor Fragmentation

- increases TLB pressure
- increases walk depth

### Poor Cacheability

- increases memory latency
- reduces bandwidth

### Stage-2 Overhead

- doubles translation cost
- increases TLB pressure

### SMP Coherency Traffic

- increases interconnect load
- increases latency

**********

**********

# 19.3 — Optimization Principle #1: Use Block Mappings

**********

**********

Block mappings reduce:

- TLB pressure
- page-table depth
- walk cache pressure
- memory traffic

### Block Sizes (4 KB granule)

- L1 block → **1 GB**
- L2 block → **2 MB**

### Best Practice

Use blocks for:

- kernel code/data
- large contiguous RAM regions
- guest physical memory (IPA)
- Stage-2 mappings

### Avoid

Using pages for large regions.

**********

**********

# 19.4 — Optimization Principle #2: Minimize Page-Table Depth

**********

**********

Page-table depth depends on:

- granule size
- VA size
- TCR configuration

### Best Practice

Use **48-bit VA** with **4 KB granule**:

- L0 unused
- L1 → L2 → L3
- minimal depth
- minimal walk cost

### Avoid

Using 52-bit VA unless required.

**********

**********

# 19.5 — Optimization Principle #3: Use Write-Back Cacheability

**********

**********

Normal WB (0xFF) provides:

- highest performance
- lowest latency
- best bandwidth
- best SMP behavior

### Best Practice

Use WB for:

- RAM
- code
- data
- DMA buffers

### Avoid

Using NC or WT unless required.

**********

**********

# 19.6 — Optimization Principle #4: Use Inner Shareable

**********

**********

Inner Shareable ensures:

- L1/L2 coherence
- TLB broadcast
- walk cache coherence
- SMP correctness

### Best Practice

Use Inner Shareable for:

- all normal memory
- DMA buffers
- shared memory

### Avoid

Non-Shareable for RAM.

**********

**********

# 19.7 — Optimization Principle #5: Reduce TLB Pressure

**********

**********

TLB pressure increases when:

- too many pages
- too many small mappings
- too many ASIDs
- too many VMIDs
- too many Stage-2 mappings

### Best Practice

- use blocks
- reduce fragmentation
- use global kernel mappings
- reuse ASIDs carefully
- reuse VMIDs carefully

**********

**********

# 19.8 — Optimization Principle #6: Optimize Descriptor Layout

**********

**********

### Best Practice

- align blocks properly
- avoid mixing block/page mappings
- avoid sparse page tables
- keep kernel mappings contiguous
- keep Stage-2 mappings contiguous

### Avoid

Fragmented descriptor layouts.

**********

**********

# 19.9 — Optimization Principle #7: Optimize Stage-2

**********

**********

Stage-2 doubles translation cost.

### Best Practice

- use blocks for IPA → PA
- minimize Stage-2 fragmentation
- minimize Stage-2 page count
- use WB cacheability
- use Inner Shareable

### Avoid

Using pages for large IPA regions.

**********

**********

# 19.10 — Optimization Principle #8: Optimize TLB Invalidation

**********

**********

TLBI is expensive.

### Best Practice

- invalidate by ASID when possible
- invalidate by VA when possible
- avoid ALLE1/ALLE2 unless required
- batch TLBI operations
- use generation-based ASID/VMID allocation

**********

**********

# 19.11 — Optimization Principle #9: Optimize Cache Maintenance

**********

**********

Cache maintenance is expensive.

### Best Practice

- avoid unnecessary DC operations
- avoid cleaning entire cache
- avoid invalidating entire cache
- use VA-based operations
- batch cache maintenance

### Avoid

Cleaning MMIO.

**********

**********

## 19.12 — Optimization Principle #10: Optimize DMA Interaction

**********

**********

DMA performance depends on:

- cacheability
- shareability
- alignment
- cache maintenance

### Best Practice

- use Normal WB for DMA buffers
- use Inner Shareable
- clean before DMA read
- invalidate after DMA write

**********

**********

# 19.13 — Optimization Principle #11: Optimize SMP Behavior

**********

**********

SMP performance depends on:

- shareability
- cacheability
- TLB broadcast
- walk cache coherence

### Best Practice

- use Inner Shareable
- avoid Non-Shareable RAM
- avoid excessive TLBI
- avoid excessive cache maintenance

**********

**********

## 19.14 — Optimization Principle #12: Optimize Hypervisor Behavior

**********

**********

Hypervisor performance depends on:

- Stage-2 design
- VMID allocation
- TLB behavior
- cache maintenance

### Best Practice

- use blocks for Stage-2
- minimize Stage-2 fragmentation
- batch TLBI operations
- batch cache maintenance

**********

**********

# 19.15 — Optimization Principle #13: Optimize QNX/RTOS Behavior

**********

**********

QNX requires:

- deterministic performance
- minimal fragmentation
- minimal TLBI
- minimal cache maintenance

### Best Practice

- use blocks for partitions
- use static ASID ranges
- use static VMID ranges
- avoid dynamic remapping

**********

**********

# 19.16 — Optimization Principle #14: Optimize Page-Table Memory Placement

**********

**********

Page tables should be:

- in Normal WB memory
- Inner Shareable
- contiguous
- aligned

### Avoid

Placing page tables in:

- Device memory
- Non-Shareable memory
- Non-cacheable memory

**********

**********

## 19.17 — Optimization Principle #15: Optimize Walk Cache Behavior

**********

**********

Walk cache accelerates page-table walks.

### Best Practice

- use blocks
- reduce page-table depth
- reduce fragmentation
- use WB cacheability
- use Inner Shareable

**********

**********

# 19.18 — Optimization Principle #16: Optimize Instruction Fetch

**********

**********

Instruction fetch performance depends on:

- I-cache behavior
- TLB behavior
- descriptor layout

### Best Practice

- use blocks for code
- use WB cacheability
- use Inner Shareable
- avoid fragmentation

**********

**********

# 19.19 — Optimization Principle #17: Optimize Data Access

**********

**********

Data access performance depends on:

- D-cache behavior
- TLB behavior
- descriptor layout

### Best Practice

- use blocks for data
- use WB cacheability
- use Inner Shareable
- avoid fragmentation

**********

**********

## 19.20 — Chapter Summary

**********

**********

This chapter covered:

- MMU performance bottlenecks
- block mapping optimization
- page-table depth optimization
- cacheability optimization
- shareability optimization
- TLB pressure reduction
- descriptor layout optimization
- Stage-2 optimization
- TLBI optimization
- cache maintenance optimization
- DMA optimization
- SMP optimization
- hypervisor optimization
- QNX/RTOS optimization
- walk cache optimization
- instruction/data access optimization